

Repeated Reed–Solomon Openings Break Sigurd

Few-Signature Witness Recovery and Fresh-Message Forgery Against All Three Submitted Parameter Sets

Martin Feussner
Selmer Center, University of Bergen
`martin.feussner@uib.no`

25 September 2026

Provenance and review status. The attack, derivation, implementation, experiments, and initial manuscript were produced independently by OpenAI Codex using Daybreak Blue at max reasoning effort during an AI-run audit of NGCC signature candidates. At the date above, no human cryptanalyst has independently verified the argument, implementation, measurements, or novelty assessment. This report is a preliminary disclosure for human review and reproduction.

Abstract

Sigurd is an NGCC submission whose signing witness is a regular-syndrome- decoding vector. Its vector-commitment layer is intended to hide that witness by appending fresh random field elements and applying one Reed–Solomon encoding. Both submitted implementations instead pad the input to a multiple of 32 and encode 32 disjoint short chunks. Most early chunks contain only the long-term witness and no fresh padding. A signature explicitly opens selected encoded-witness symbols, so openings from different signatures repeat within those chunks.

We give a passive key-recovery attack. For each witness-only chunk, the attacker collects enough distinct public evaluations across signatures and inverts a small Vandermonde matrix. This recovers nearly the entire one-hot witness. Subtracting its contribution from the public syndrome leaves a thin binary linear system that recovers the remaining blocks. The recovered witness is an equivalent signing key and can run the public prover algorithm on arbitrary new messages. The attack does not exploit the placeholder hash, a side channel, malformed inputs, or a rare secret-key event.

End-to-end experiments used the unmodified submitted reference verifiers. We recovered and forged for 20 of 20 Sigurd-128 keys with 4–5 signatures, 10 of 10 Sigurd-256 keys with 5–7 signatures, and 5 of 5 Sigurd-512 keys with 7–8 signatures. Mean query counts were 4.5, 6.4, and 7.2. Total single-threaded CPU time, including generation and verification of every queried signature and an accepted fresh-message forgery, was 0.114–0.137 seconds, 1.077–1.432 seconds, and 14.198–16.180 seconds, respectively, on an Intel Core i5-6500. Representative peak resident memory was 14.8, 56.3, and 249.5 MB. Thus every submitted security level is practically broken on an ordinary laptop.

1 Introduction

Sigurd is a post-quantum signature candidate submitted to the Next-generation Commercial Cryptographic Algorithms Program (NGCC). It proves knowledge of a regular-syndrome-decoding (RSD) witness using a VOLE-in-the-head construction and a Reed–Solomon/Merkle vector commitment [1]. The public key contains a binary matrix seed and a syndrome $y = He$. The secret witness e is divided into blocks of six bits, with exactly one bit set in every block.

The vector-commitment layer is central to the claimed zero knowledge. The specification appends fresh random padding to the lifted witness and describes one global Reed–Solomon encoding. Random padding is supposed to make the few opened codeword positions statistically hide the unpadded input. The submitted C implementations use a different concrete map: they partition the padded coefficient vector into 32 chunks and encode every chunk independently. The fresh padding is at the end, so many chunks consist entirely of the long-term witness. Their codewords are identical in every signature.

This turns ordinary signatures into an erasure-decoding oracle. Each signature serializes the opened encoded-witness values themselves. Once an attacker has seen k distinct positions of a length- s chunk, interpolation recovers all k input symbols. Applying this independently to the witness-only chunks recovers all but 9, 17, and 10 one-hot blocks in the three parameter sets. The public equation $He = y$ then determines the remaining 54, 102, and 60 bits by Gaussian elimination.

Our contributions are:

- We identify a deterministic privacy failure in the concrete Reed–Solomon map used by both submitted implementations.
- We give an exact, polynomial-time witness-recovery algorithm from a handful of ordinary signatures. It needs neither chosen messages nor access to signing randomness.
- We prove that k distinct openings in every witness-only chunk plus a full-rank residual public matrix suffice for complete recovery.
- We turn the recovered witness into an equivalent signing key and produce signatures on fresh messages that the unmodified submitted verifiers accept.
- We validate the attack across 35 independently generated keys and all three submitted security levels, with at most eight signatures and less than 17 CPU-seconds in every trial.

The result is a full EUF-CMA break of the submitted implementations, rather than signature malleability or a distinguisher. The abstract construction with one correctly implemented global Reed–Solomon code is outside the claim of this attack; Section 6 separates the two precisely.

2 Relevant parts of Sigurd

We use the submission’s notation where possible. Only the key relation, witness lifting, vector commitment, and serialized openings are needed for the attack.

2.1 The RSD signing witness

For every submitted level, the public key defines

$$H \in \mathbb{F}_2^{r \times m}, \quad y = He \in \mathbb{F}_2^r.$$

The secret vector is split into $n = m/6$ blocks

$$e = (e_0 \parallel \cdots \parallel e_{n-1}), \quad e_i \in \mathbb{F}_2^6, \quad \text{wt}(e_i) = 1.$$

If the nonzero position of e_i is $a_i \in \{0, \dots, 5\}$, Sigurd lifts the block to

$$\hat{e}_i = Y^{a_i}.$$

The implementation stores an element of the ambient extension representation as τ coefficients over $\mathbb{K} = \mathbb{F}_{2^{16}}$. Therefore a lifted secret block is simply a raw coefficient vector with one in coordinate a_i , zeros in the other first six coordinates, and zeros in coordinates $6, \dots, \tau - 1$.

For each signature the signer appends a fresh augmentation tail of length T to obtain

$$\hat{e}' = (\hat{e}_0, \dots, \hat{e}_{n-1}, a_0, \dots, a_{T-1}) \in \mathbb{E}^{N'}, \quad N' = n + T.$$

The public key is reusable and the prefix of length n is fixed; only the tail is refreshed.

2.2 What a signature opens

The signer encodes $\hat{e}'$ and a fresh mask vector, places pairs of encoded symbols in Merkle trees, and derives Q distinct opening positions from the Fiat–Shamir transcript. The serialized signature ends with

$$\text{Enc}(\hat{e}')_{j_0}, \dots, \text{Enc}(\hat{e}')_{j_{Q-1}},$$

together with authentication hashes. The verifier reconstructs the masking symbols from the public masked word and these encoded-witness values. Thus the witness-code symbols are transmitted in the clear. Their positions are also public: anyone can replay the transcript hash and the deterministic without-replacement sampler.

This disclosure is safe only if each queried code symbol is hidden by the fresh augmentation. That property holds for the intended global encoding, but it fails for the submitted chunked encoder.

2.3 The concrete 32-way encoder

Let

$$k = \left\lceil \frac{N'}{32} \right\rceil$$

and pad $\hat{e}'$ with zeros to length $32k$. Both the reference and optimized implementations process 32 chunks

$$(\hat{e}'_{ck}, \dots, \hat{e}'_{ck+k-1}), \quad 0 \leq c < 32,$$

independently. For each raw coefficient lane $d \in \{0, \dots, \tau - 1\}$ they form

$$F_{c,d}(X) = \sum_{b=0}^{k-1} \hat{e}'_{ck+b,d} X^b \in \mathbb{K}[X] \quad (1)$$

and output evaluations at

$$x_j = \alpha^{j+1}, \quad 0 \leq j < s, \quad s = \rho^{-1}k,$$

where $\alpha = 2$ generates $\mathbb{K}^*$. Global codeword position $cs + j$ contains all τ values $F_{c,d}(x_j)$. The implementation emits the first N positions of the resulting 32 codewords.

This map is a direct sum of short Reed–Solomon evaluation maps. It is not the single degree- $< N'$ polynomial evaluation described by the specification. In particular, if

$$c < C := \left\lfloor \frac{n}{k} \right\rfloor,$$

then all k inputs to chunk c belong to the fixed witness prefix. No fresh augmentation symbol enters Equation (1), so the entire chunk repeats under key reuse.

Table 1 gives the resulting attack dimensions. The column $R = n - Ck$ is the number of secret blocks left outside the completely fixed chunks.

3 Key recovery

The attack has two algebraic stages. It first interpolates the repeated witness-only chunks and then completes the short unknown suffix using the public syndrome.

Table 1: Submitted parameters and chunk dimensions used by the attack.

Instance	m	n	N'	N	Q	τ	k	s	C	R
Sigurd-128	1302	217	392	1568	160	10	13	52	16	9
Sigurd-256	2748	458	651	5208	171	16	21	168	21	17
Sigurd-512	5676	946	1239	19824	256	32	39	624	24	10

3.1 Interpolating a fixed chunk

For a fixed chunk $c < C$, collect openings at k distinct local indices $j_0, \dots, j_{k-1}$. For each raw lane d , the public values satisfy

$$\begin{pmatrix} F_{c,d}(x_{j_0}) \\ \vdots \\ F_{c,d}(x_{j_{k-1}}) \end{pmatrix} = \begin{pmatrix} 1 & x_{j_0} & \cdots & x_{j_0}^{k-1} \\ \vdots & \vdots & & \vdots \\ 1 & x_{j_{k-1}} & \cdots & x_{j_{k-1}}^{k-1} \end{pmatrix} \begin{pmatrix} \hat{e}'_{ck,d} \\ \vdots \\ \hat{e}'_{ck+k-1,d} \end{pmatrix}.$$

The evaluation points are distinct, so this Vandermonde matrix is invertible over $\mathbb{K}$. One inversion recovers all τ lanes and hence all k lifted witness blocks in the chunk.

Lemma 1. *For every $c < C$, any k distinct opened positions in chunk c uniquely determine $\hat{e}'_{ck}, \dots, \hat{e}'_{ck+k-1}$.*

Proof. Equation (1) has degree less than k . Evaluation at k distinct field points is a bijection from its coefficient vector to its value vector. The same Vandermonde matrix applies independently to every raw lane. $\square$

After all C chunks have been decoded, the attacker has Ck of the n secret blocks. The expected number of distinct local positions seen in one chunk after t independent signatures is

$$s \left(1 - \left(1 - \frac{Q}{N} \right)^t \right), \quad (2)$$

because a fixed global position is opened with probability Q/N in each signature. The attacker simply continues until every fixed chunk has at least k distinct positions; this is observable and requires no guess.

3.2 Completing the suffix from the public key

Let e_K denote the recovered $6Ck$ prefix bits and let e_U contain the remaining $u = 6R$ bits. Partition the public matrix accordingly:

$$H = (H_K \mid H_U).$$

The public equation gives

$$H_U e_U = y + H_K e_K. \quad (3)$$

For the submitted levels, u is only 54, 102, and 60. Binary Gaussian elimination therefore recovers e_U immediately. Each resulting six-bit block can be checked for weight one, and the complete vector can be verified against $He = y$.

For a uniformly random $r \times u$ binary matrix with $r \geq u$, the exact full-column-rank probability is

$$\prod_{i=0}^{u-1} (1 - 2^{i-r}),$$

which is overwhelmingly close to one at the submitted dimensions. If a rare key gives a rank deficiency, the attacker solves the small affine space and uses the one-hot conditions; the observed attack is therefore not based on a special key property.

3.3 From a witness to a forgery

The serialized secret key stores a seed that regenerates e , so witness recovery need not reproduce the secret-key byte string. The signing prover, however, uses only the expanded public matrix, public syndrome, and witness vector. An attacker can lift the recovered e , sample fresh augmentation and masking values, and execute the public signing protocol exactly as an honest signer. This is an equivalent signing key.

Theorem 2. *Suppose the attacker has collected k distinct openings in every chunk $c < C$, and the residual system in Equation (3) has a unique solution. Then the attacker recovers the complete Sigurd signing witness and can generate an accepted signature on any fresh message in polynomial time.*

Proof. The lemma recovers the first Ck lifted blocks exactly. Their one-hot raw representations give the corresponding binary prefix e_K . The unique solution of Equation (3) is the remaining witness suffix e_U , so the concatenation is the complete valid RSD witness. Sigurd’s prover is a proof of knowledge whose honest algorithm takes that witness and public instance as input. Running it with fresh randomness produces a complete transcript satisfying every verifier equation. Fiat–Shamir challenge values are recomputed from the new message and transcript, so the resulting signature is a fresh-message forgery rather than a replay. $\square$

4 Complete attack algorithm

The following procedure uses only the public key and valid signatures under one key.

1. Parse each signature and replay its public Fiat–Shamir hash chain to derive the Q distinct opening positions.
2. Pair each position with the corresponding encoded-witness value in the final signature field. Write the position as $cs + j$, with chunk c and local evaluation index j .
3. For every $c < C$, retain one value for each distinct j . Repeated observations must agree; disagreement detects a parsing error or a signature from another key.
4. Continue collecting signatures until every fixed chunk contains at least k distinct local positions.
5. For each fixed chunk, choose any k positions, form the Vandermonde matrix (x_j^b) , and solve it over $\mathbb{F}_{2^{16}}$ for all τ raw lanes.
6. Decode each recovered lifted value as a one-hot six-bit block. This yields e_K .
7. Expand the public matrix from its public seed, compute the residual in Equation (3), and solve the $r \times 6R$ binary system. Check the one-hot conditions and $He = y$.
8. Use the recovered witness and fresh randomness to run Sigurd’s public prover on a new message. Serialize the transcript normally.

No step queries or predicts a random oracle. The hash is used only to recover positions that the verifier also derives. The dominant algebra consists of small Vandermonde and binary systems.

5 End-to-end experiments

5.1 Methodology

We tested the official NGCC submission archive with SHA-256 digest

443ab14257658d89a60e14ad0d4b7f4df9fcdd06da4f42fcb29bbbf7c75a97153.

The experiments compiled the submitted reference sources with GCC 13.3.0 and `-O3` on 64-bit Linux. The machine had an Intel Core i5-6500 CPU at 3.20 GHz, four physical cores, and 16 GiB of RAM. Every reported process was single threaded.

For each independently seeded key, we requested signatures on distinct short messages and verified every genuine signature before feeding it to the attacker. Collection stopped as soon as every witness-only chunk had k distinct symbols. The attack then interpolated the fixed chunks and solved the public residual system. Comparison with the signer-generated witness was performed only after recovery and was not an attack input. Finally, a signing transcript was generated directly from the recovered witness for a message that had never been queried, and the unmodified submitted verification API was called on that public key, message, and signature.

The timed region included generation and verification of every queried signature, parsing, key recovery, construction of the forgery, and verification of the forgery. A real passive attacker can omit verification of the genuine signatures, so these timings are conservative. Peak memory includes the submission’s expanded public matrices and verifier workspaces.

5.2 Results

Table 2 reports all trials. Every recovered prefix decoded to valid one-hot blocks, every residual matrix had full column rank, every recovered witness matched the test secret exactly, and every fresh-message forgery was accepted.

Table 2: End-to-end key recovery and fresh-message forgery results.

Instance	Keys	Success	Signatures	Mean	CPU seconds	Peak RSS
Sigurd-128	20	20/20	4–5	4.50	0.114–0.137	14.8 MB
Sigurd-256	10	10/10	5–7	6.40	1.077–1.432	56.3 MB
Sigurd-512	5	5/5	7–8	7.20	14.198–16.180	249.5 MB

The signature counts match the coverage calculation. For example, after four Sigurd-128 signatures Equation (2) predicts 18.3 distinct symbols per 52-symbol chunk, while only 13 are needed. For Sigurd-256 after six signatures it predicts 30.5 of 168 symbols, with 21 needed. For Sigurd-512 after seven signatures it predicts 54.3 of 624 symbols, with 39 needed. The stopping rule depends on the least-covered chunk, which explains the small variation between keys.

Attack storage for the collected fixed codewords is about 17 KB, 113 KB, and 959 KB for the three levels. The larger measured resident sets come from expanding the submitted binary matrix as arrays of machine words and from running the reference verifier; a streaming public-matrix expansion would substantially reduce them.

6 Scope of the security argument

The Sigurd specification defines a Reed–Solomon encoder as evaluation of one polynomial whose coefficient vector contains the full padded input [1, Sec. 2.5 and Sec. 4.1]. It states that randomized padding hides the unpadded inputs at queried positions. In a global encoding, every evaluation depends on the fresh tail, so openings from separate signatures do not repeat.

The submitted implementations instead expose a direct sum of 32 encoders. Random padding can hide only chunks that actually contain padding. For $c < C$, the opened value is a deterministic linear function of the secret prefix alone. The special-honest-verifier zero-knowledge intuition therefore fails under key reuse, and the specification explicitly permits at least 2^{64} messages per key while the attack uses at most eight.

This distinction affects the scope of the result:

- The reference and optimized implementations for all three submitted levels use the vulnerable chunk map, so their valid signatures leak the witness and admit end-to-end forgery.
- The attack is independent of the concrete placeholder hash and remains valid if that hash is replaced, because opening positions and values must remain public to the verifier.
- The attack does not show that a faithful monolithic $[N, N']$ Reed–Solomon implementation of the abstract vector commitment leaks in the same way.

A repair must restore fresh masking to every opened linear form. Implementing the specified global encoding is the direct approach. Alternatively, a chunked design would need fresh private padding inside every chunk and a new privacy analysis that covers accumulation across the advertised number of signatures. Changing only the query count, Merkle layout, or hash function does not remove the repeated-codeword leakage.

7 Independent reproduction procedure

The following procedure is sufficient to reproduce the result without any attack artifact from the discovery environment.

1. Obtain the official Sigurd submission archive and verify the SHA-256 digest printed in Section 5. Build a reference instance and its normal key-generation, signing, and verification interfaces.
2. Generate one key pair. Obtain valid signatures on successively numbered messages under the same key.
3. Parse the fields in the order specified by the submission: Merkle roots; two commitment evaluations; the interactive response arrays; six final-response elements; N' masked-word elements; a little-endian authentication-hash count and those hashes; and finally Q opened encoded-witness elements.
4. Replay the documented Fiat–Shamir chain. In particular, derive the opening seed from the message hash, the polynomial-response hash, and the serialized masked word, then run the submission’s public without-replacement sampler. This yields the global position attached to every final opened value.
5. Compute $k = \lceil N'/32 \rceil$, $s = \rho^{-1}k$, and $C = \lfloor n/k \rfloor$. For positions below Cs , group openings by $c = \lfloor j/s \rfloor$ and local index $j \bmod s$. Stop when each group has k distinct indices.
6. Use evaluation points $x_j = 2^{j+1}$ in the submitted $\mathbb{F}_{2^{16}}$ representation. Invert the $k \times k$ Vandermonde system for every group and apply the inverse to all raw lanes. Decode the resulting monomials $1, Y, \dots, Y^5$ as one-hot blocks.
7. Expand H and y from the public key. Remove the contribution of the recovered prefix, solve Equation (3) over $\mathbb{F}_2$, and verify regularity and the complete public syndrome.

8. Run the normal prover with this recovered witness and fresh randomness on a new message. Acceptance by the unmodified verification API is the end-to-end forgery test.

Useful diagnostic checks are deterministic. A repeated global position in a fixed chunk must carry the same raw value across signatures. Interpolated secret blocks must be one-hot in the first six lanes. The completed witness must satisfy $He = y$. Failure of any check indicates a transcript-layout, endianness, or position-sampler mismatch rather than a statistical failure of the attack.

8 Conclusion

The submitted Sigurd encoders partition one padded vector into 32 independent Reed–Solomon codewords. Fresh padding occupies only late chunks, while the signature publicly opens symbols from every chunk. Key reuse therefore reveals repeated evaluations of secret-only polynomials. Small Vandermonde systems recover nearly the whole witness, and the public syndrome supplies the short suffix.

The result is practical at every submitted level: 4–8 ordinary signatures, less than 17 seconds of single-threaded CPU time, and less than 250 MB of measured memory. The recovered witness signs arbitrary new messages, and the submitted verifiers accepted all 35 experimental forgeries. Independent human review is still required, but the present evidence is a complete, reproducible key-recovery and forgery chain.

References

- [1] Y. Deng, Y. Wei, X. Zhang, Z. Wu, Z. Zhang, and L. Qin, “Sigurd: Algorithm Specifications,” Version 1.0, NGCC submission, 29 June 2026.
[Official NGCC submission archive.](#)
- [2] I. S. Reed and G. Solomon, “Polynomial Codes over Certain Finite Fields,” *Journal of the Society for Industrial and Applied Mathematics*, vol. 8, no. 2, pp. 300–304, 1960.